

ممثل
قسم هندسة الطاقة الكهربائية
محمد حتاحت

أنظمة المعالج الدقيق

- اسمبلي -

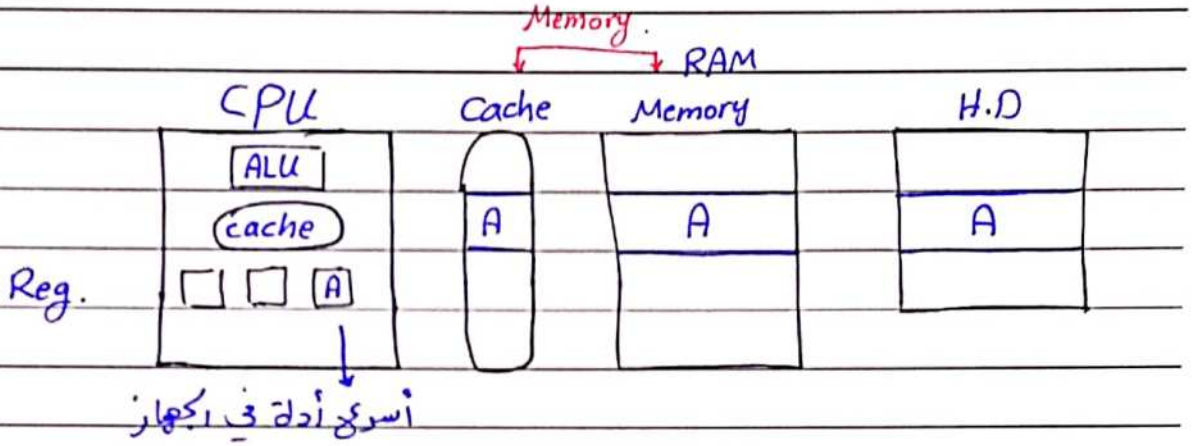
اعداد : عادل البشير

طلاب البور والإتصالات
spark team
spark-fet.com

Scanned by CamScanner

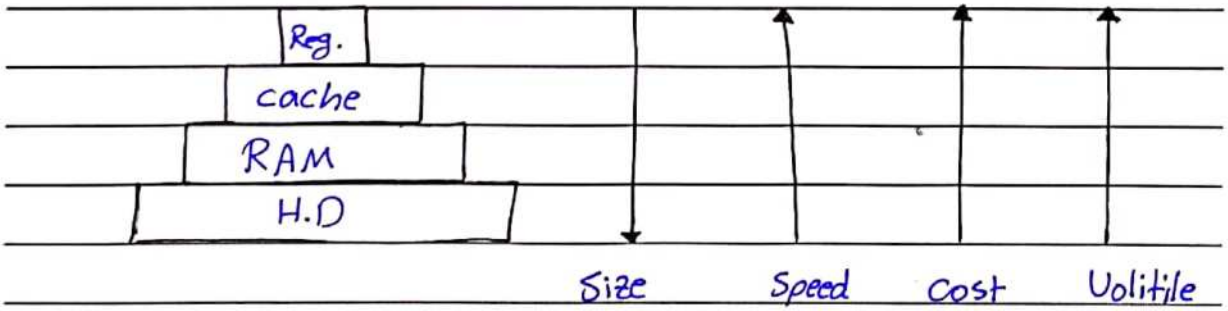
Assembly

مقدمة



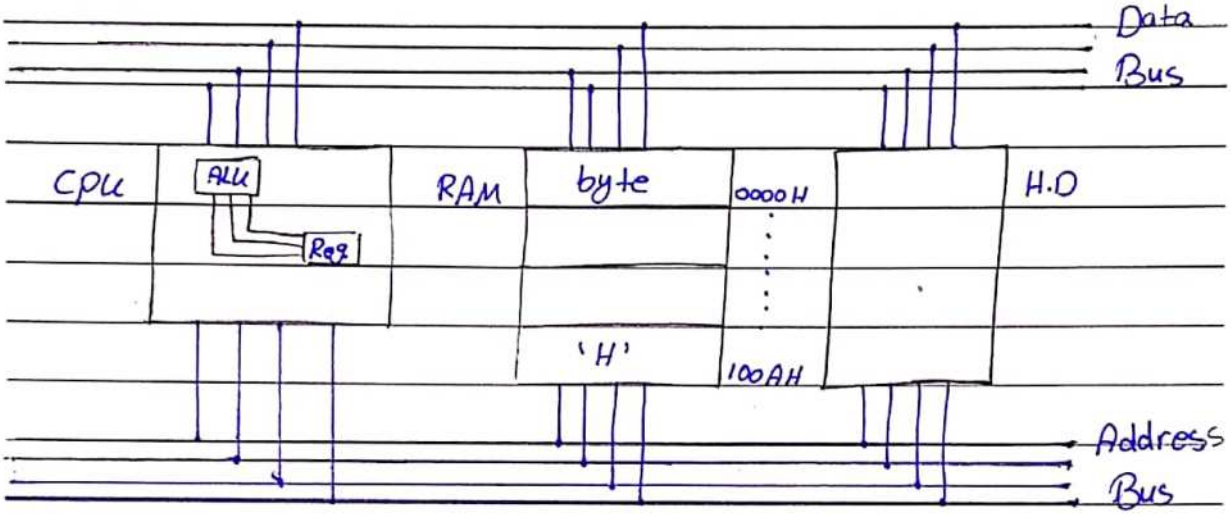
* لو عندي ملف أسمه A في الـ H.D حسب حجمه يوخذ مساحة في الـ Memory
فمثلاً حجمه 1byte يوخذ 1byte وهكذا ، وتسمى هذه العملية :
Caching. عملية نقل الـ Data من الأقل سرعة إلى الأعلى سرعة في الـ

* level 1 cache ← أسرع من level 2 cache ولكنها أكثر سرعة.
* level 2 cache ← أسرع من level 3 cache ولكنها أكثر سرعة وهكذا .



Scanned by CamScanner
Scanned by CamScanner

Adel Albasheer



* لازم يكون في ترابط بين حجم الـ RAM و الـ Address Bus الموجودة
كل ما زاد الـ Address Bus size بتزيد الـ Memory Size وبتزيد السرعة
وبقل رجوعي لـ H.D و بتزيد الـ performance

Memory Size = 2^n , where: n : # of Address Bus.

Scanned by CamScanner

Scanned by CamScanner

* مراجعة مهمة :-

* First Complement :-

بنعكس الرقم مباشرة

Ex:- 11001100 $\xrightarrow[1's\ comp.]{}$ 00110011

* Second Complement :-

بنعكس الرقم عند اول one

Ex:- 11001100 $\xrightarrow[2's\ comp.]{}$ 00110100

* Ascii Code :-

* كل Char جمعه 1byte هو نظام يستخدم الرموز (Char.)

'A' = 41 H 'a' = 61 H '1' = 31 H

'B' = 42 H 'b' = 62 H '2' = 32 H $ = 24 H

'C' = 43 H 'c' = 63 H '9' = 39 H

* انواع العد :-

① Binary. ← ثنائي (2)

② Decimal ← عشري (10)

③ Octal ← ثماني (8)

④ Hexadecimal ← سادس عشر. (16)

Scanned by CamScanner

Scanned by CamScanner

* التحويل من أنظمة العد المختلفة إلى النظام العشري :-

ملاحظات :-

1- النظام يستوعب أرقام لحد العدد 9.

2- هو يرجع لجميع أنظمة العد.

Ex:- $(10111)_2 \longrightarrow (\quad)_{10}$

1 0 1 1 1

$2^4*1 + 2^3*0 + 2^2*1 + 2^1*1 + 2^0*1$

$16 + 0 + 4 + 2 + 1 = (23)_{10}$

Ex:- $(43)_8 \longrightarrow (\quad)_{10}$

$8^1*4 + 8^0*3$

$32 + 3 = (35)_{10}$

Ex:- $(BA)_{16} \longrightarrow (\quad)_{10}$

$16^1*11 + 16^0*10$

$176 + 10 = (186)_{10}$

in hexa ⇒ A = 10
B = 11
C = 12
D = 13
E = 14
F = 15

ملاحظة:- أن طريقة التحويل ثابتة لجميع أنظمة العد.

Scanned by CamScanner

Scanned by CamScanner

* التحويل من النظام العشري الى انظمة العد المختلفة :-

ملاحظة:- هنا في التحويل من عشري الى اي نظام آخر نتعامل مع الرقم كامل مثلاً (173) نأخذه كامل دون اي تجزئة وهنا فقط في حالة من عشري الى اي نظام .

Ex:- $(94)_{10} \longrightarrow (\quad)_2$

$2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

64 32 16 8 4 2 1

1 0 1 1 1 1 0 $\Rightarrow (1011110)_2$

Ex:- $(89)_{10} \longrightarrow (\quad)_8$

الطريقة الاولى

$8^2\ 8^1\ 8^0$

64 8 1

$(1\ 3\ 1)_8$

الطريقة الثانية

8	89
8	11 1
8	1 3
	0 1 ↑

$(131)_8$ ← للخلف

Ex:- $(210)_{10} \longrightarrow (\quad)_{16}$

$16^1\ 16^0$

16 1

13 2 ←

	13
16	210
	208
	2

↳ $(132)_{16}$ X $\Rightarrow (D2)_{16}$ ✓

Scanned by CamScanner

Scanned by CamScanner

* التحويل بين الأنظمة (الثنائي، الثماني و السادس عشر)

ملاحظات :-

1- في هذا التحويل دائماً نرجع إلى النظام العشري بعدها نحول إلى النظام المراد التحويل إليه
2- هناك بعض الحالات لا تحتاج التحويل إلى النظام العشري.

Ex:- $(67)_8 \longrightarrow (\quad)_2$

$8^1*6 + 8^0*7$

$48 + 7 = (55)_{10}$

55

↳ 32 16 8 4 2 1

$(1\ 1\ 0\ 1\ 1\ 1)_2$ #

طريقة أخرى.

$(67)_8$

$2^2\ 2^1\ 2^0 \quad 2^2\ 2^1\ 2^0$

4 2 1 4 2 1

1 1 0 . 1 1 1

$\Rightarrow (110111)_2$

Ex:- $(110111)_2 \longrightarrow (\quad)_8$

$2^2*1 + 2^1*1 + 2^0*0 \quad 2^2*1 + 2^1*1 + 2^0*1$

$4 + 2 + 0 \quad 4 + 2 + 1$

$6 \quad 7$

$\Rightarrow (67)_8$

* الداعي للتحويل للعشري، نحل على طول لأنه يوجد علاقة بينهم.

⊗ اذا كان من ثنائي إلى سادس عشر نأخذ كل 4 digit

Scanned by CamScanner

Scanned by CamScanner

Adel Albasheer

* عنا نوعين من CPU

① 8088 microprocessor

② 8086 microprocessor.

* الفرق بينهم :-

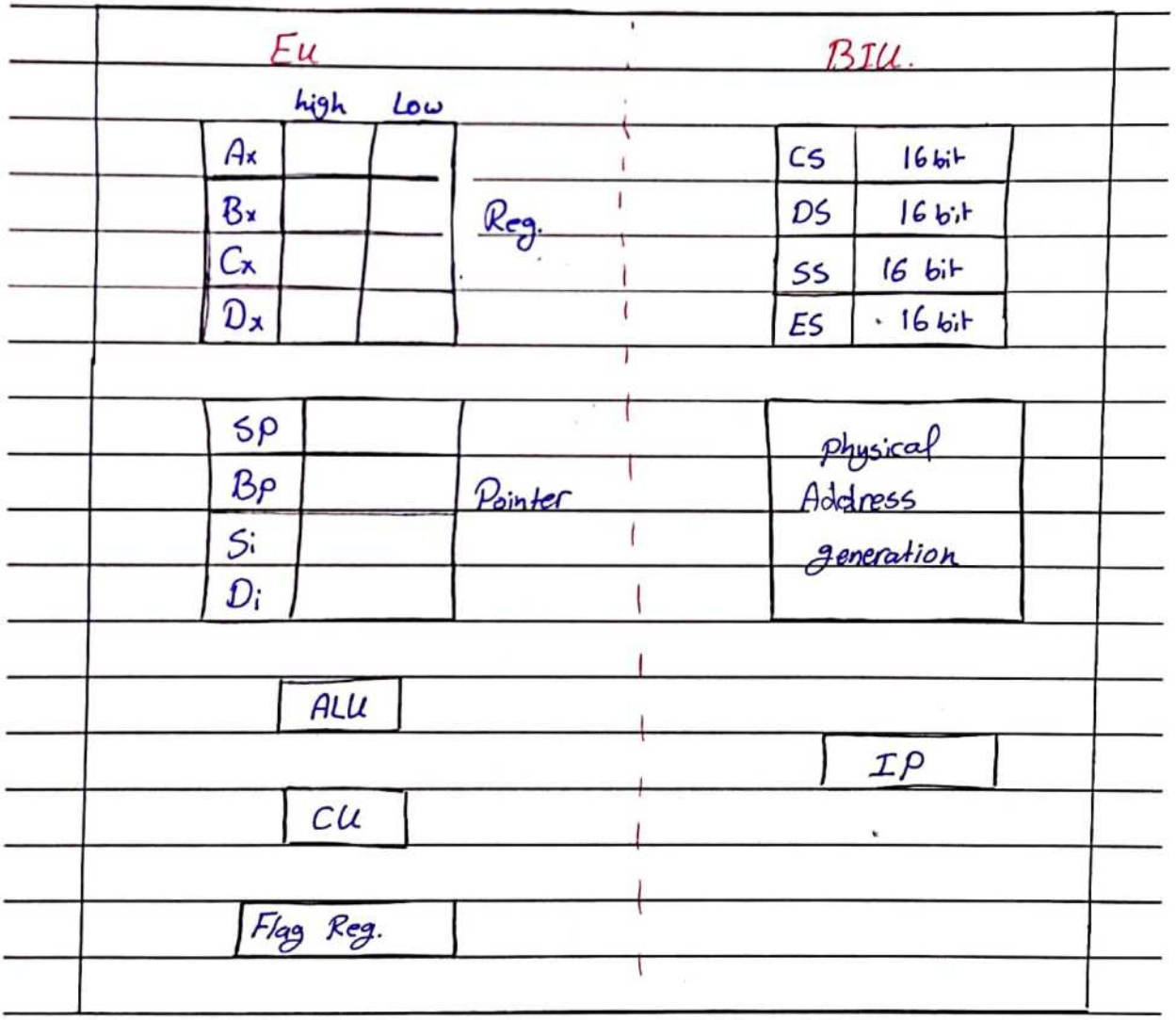
	8086	8088
Transt	29000	29000
CPU (MHz)	4.77 - 10	4.77 - 8
Reg. size	16 bit	16 bit
Data Bus	16 bit	8 bit
Address Bus	20 bit	20 bit

* ممكن يجي عليه سؤال ضمن دائرة.
قصدو بحكيلك كم Reg. size لـ 8086 ؟

Scanned by CamScanner

Scanned by CamScanner

* Microprocessor 8086 :-



EU :- Execution Unit

BIU :- Bus Interface Unit.

Scanned by CamScanner

Scanned by CamScanner

Adel Albasheer

* أجزاء الـ microprocessor 8086 :-
- Registers
- Pointer
- ALU
- CU
- Flag Reg.
- IP
- physical Address generation.

* Register :-

يوجد لدينا 4 وحدة من الـ Reg كل واحد يحتوي على 16bit

Reg حيث ينقسم الـ إلى نصفين :-
High → H
Low → L

Ax →	AH	AL
	8 bit	8 bit
Bx →	BH	BL
	8 bit	8 bit
Cx →	CH	CL
	8 bit	8 bit
Dx →	DH	DL
	8 bit	8 bit

Scanned by CamScanner

Scanned by CamScanner

Ex :-

Mov Ax , 400H

ملاحظة :-

① MOV :- تستخدم لتخزين لقيم داخل المتغيرات او الـ Reg .

هون رح استخدم كامل الـ Reg كونه عبارة عن 12 bit

② عندما يكون النظام بالـ hexa يكون كل حد عبارة عن 4bit

مثال :- 400 H
4bit 4bit 4bit

Mov AL , 40 H

هون رح استخدم نص الـ Reg كونه 8bit و بستخدم قسم LOW

③

	H	L
Ax →	AH	AL

- بخزن أول اشي في الـ LOW ولما يتجاوز الـ 8bit بيلش أخزن بالـ High .

* Instruction Pointer :-

تستخدم في تحديد قيمة الـ offset في الـ memory وتأشر على قيم المخزنة فيها .

SP :- Stack Pointer — SP مربوط مع → SS

BP :- Base Pointer — BP مربوط مع → SS

Si :- Index Reg. — Si مربوط مع → DS

Di :- Index Reg. — Di مربوط مع → ES

IP :- CS — تحل قيمة الـ offset — تحل عناوين الـ Address.

Scanned by CamScanner

Scanned by CamScanner

			7000H : 00H	
70000H	inst 1	Code Seg.	قيمة الـ Logical : قيمة الـ offset	
⋮	inst 2	↕ 50H	⋮	
⋮	⋮		⋮	
70050H	inst n	Ip = 50H offset of inst n		
			7000 H:50H	inst n

* التحويل من Logical Address الى physical Address وذلك من أجل حماية وتشفير الـ Data .

Physical Address = Logical Address * 2^n + offset.

موقعه في الـ memory — يجب ان تكون مقدار القيمة بالـ hexa — لـ 20 bit دائماً قيمتها

ويجب ان يكون بالـ hexa.

n :- هي مقدار الـ Shift عند عملية التحويل و اذا لم يرد مقدار لـ Shift تكون n = 4 على اعتبار انه يحول من 16 bit الى 20 bit.

* مثلاً لو كانت 14 H بدي احولها الى 20H

وبالتالي n = 6

2^6 ⟶ 64 D ⟶ (حولها) 40 H.

Scanned by CamScanner

Scanned by CamScanner

Ex :- Ds = 5000 H , Es = 4000 H , Si = 100 H , Di = 300 H , Ip = 50 H

Calculated physical Address pointed by Di ?

Solution :-

physical Address = Es * 10H + Di

= 4000H * 10H + 300H

= 40300 H

ملاحظة :-

لم يحدد مقدار الـ Shift

وبالتالي نعتبر n=4

$2^4 \rightarrow 16D \rightarrow 10H$

لازم تحويلها الـ Hexa

ملاحظة :-

يجب الانتباه اي نوع

الـ pointer حتى يتم

اختيار عنوان الـ Address المرتبط معها.

* Flag Register :-

1 Zero Flag :- (ZF)

- Zero Result = 1
- Non Zero Result = 0

عندما يكون جواب العمليات صفر

يكون عندي الـ ZF = 1 وغير ما يكون

الناتج صفر يكون الـ ZF = 0

2 Sign Flag :- (SF)

- Negative → SF = 1
- Positive → SF = 0

اذا كان الـ bit الثامن (1) يكون سالب

وإذا (SF=1) اما اذا كان (0) يكون موجب

وإذا (SF=0)

Scanned by CamScanner

Scanned by CamScanner

3 Direction Flag :- (DF)

- From left to Right → DF = 0
- From Right to left → DF = 1

يتم تخزين او نقل اي رقم من اليمين او اليسار عن طريق تحديد قيمة الـ DF

اذا صفر ← يخزن من اليسار اي اليمين

١ ← يخزن من اليمين اي اليسار.

4 Parity Flag :- (PF)

Ex :- 11001111
01010100 +
00100011

PF = 0 ← لان عدد الـ one فردي

اذا كان عدد الواحدات (١) في الجواب

* عدد فردي فإن ← PF = 0
* عدد زوجي فإن ← PF = 1

5 Auxiliary Flag :- (AF)

عند وجود Carry ناتج من جمع الـ bit رابع من الرقم الاول مع الـ bit رابع من الرقم الثاني يكون AF = 1 ، واذا لم يظهر Carry يكون AF = 0

Ex :- 1101 0111
0000 1100 +
11100011

Carry ← ناتج عن مجموع الـ bit رابع

6 Carry Flag :- (CF)

Ex :- 11001111
01010100
00100011 CF = 1

بكل بساطة عند اجراء عملية جمع ويظهر عندي Carry يكون CF = 1 وعند عدم وجود Carry يكون CF = 0

Scanned by CamScanner

Scanned by CamScanner

[7] overflow Flag :- (OF)

تنقسم إلى :
- unsigned
- Singed

* Unsigned :-

في هذه الحالة هنا لا نهتم بإشارة الأعداد اي انها تعتبر ارقام بدون اشارة (موجبة دائماً).
حيث يتراوح مجالها (حدودها)

من Zero إلى 255 (min, max)

128	64	32	16	8	4	2	1	
0	0	0	0	0	0	0	0	min = Zero
1	1	1	1	1	1	1	1	max = 255

حدودها | overflow

Zero — 255 → فاذا تجاوزت الـ 255 يكون عندي overflow ويكون OF = 1

* Signed :-

في هذه الحالة نهتم بإشارة العدد
حيث يوجد ارقام سالبة وموجبة
ونحجز الـ bit الثامن لمعرفة نوع الرقم (سالب او موجب)
ويتراوح مجالها (حدودها)
من -128 الى 127 (min, max)

128	64	32	16	8	4	2	1	
1	0	0	0	0	0	0	0	min = -128
0	1	1	1	1	1	1	1	max = 127

حجزنا هذه الخانة لأحدد اشارة العدد

overflow | حدودها | overflow

-128 ← اذا قلت عن -128 ← OF = 1

127 ← اذا زادت عن 127 ← OF = 1

Scanned by CamScanner

Scanned by CamScanner

ملاحظة:-
اذا كان الـ bit الاخير (1) يعني سالب
لازم اعمل 2nd comp. حتى اعرف
قيمة الرقم الفعلية.

Ex:- 1100) 2nd comp.
0100

طلع قيمة الرقم 4 يعني -4

* باختصار اذا تغلبت بفهم الـ (OF) خليك على هاي القاعدة:-

* اذا جمعت رقم سالب مع رقم سالب وطلع الجواب موجب ← OF = 1
يعني عندي overflow
يعني الـ bit الاخير zero

* اذا جمعت رقم موجب مع رقم موجب وطلع الجواب سالب ← OF = 1
يعني عندي overflow
يعني الـ bit الاخير one

غير هيك يكون الـ overflow يساوي zero
OF = zero

Scanned by CamScanner

Scanned by CamScanner

* Data :-

خلينا نعرف طريقة تعريف المتغير في لغة الـ Assembly

بنتذكر في لغة الـ C++ كنا لما نعرف متغير نعمل int x = 4 وهيك بكون تعرف عندي في البرنامج قيمة x=4

أما في لغة الـ Assembly بعرف المتغيرات حسب حجمها ، يعني عندي 8bit (byte) 16bit (2 byte)

⋮

* نعرف هذه الأحجام عن طريق :-

[1] X DB 2H → Define Byte

تخزن byte فقط.

	Memory.	
هيك بصير بحجز عندي في الـ Memory		0600H
8bit (byte) وبخزن فيها قيمة X	⋮	
	⋮	
X	20	

ملاحظة :-
كل خانة في الـ Memory عبارة عن byte يعني 8bit

[2] X DW 2H → Define Word (2 byte)

DS Memory

يعني بحجز في الـ Memory مربعين

	⋮
X	00
X+1	20

[3] X DD 2H → Define Double word (4byte)
[4] X DQ 2H → Define Guard word (4 word)
[5] X DT 2H → Define Ten byte (10 byte)

Scanned by CamScanner

Scanned by CamScanner

Ex :- In Array

Var DW 4444H , 5500H , 6600H

sol

-خلينا نشوف كيف بدنا نمثلها في الـ Memory.

44	Var[0]	4444
44	Var[1]	
00	Var[2]	5500
55	Var[3]	
00	Var[4]	6600
66	Var[5]	

هيدا لون معرف متغير اسمه Var
ومخزن فيه اكثر من قيمة ، بدنا تخزن
هاي القيم في الـ Memory

ملاحظة-

① يتم قراءة الرقم عكس التخزين مع اتجاه السهم.

11 22 ← 22 11

② الأحرف لا تخزن في الـ Memory بل نستبدلها بالقيم في الـ Ascii Code

Ex :- X DW 'A'

sol

A = 41H
Null

هون بده
لحيز حريعين
يعني 2byte
لانه DW

Ex :- X DW 'AB'

sol

41H
Null
42H
Null

هون لما يكون عندي ارقام
وأحرف بين (' ')
بنعتبرها نفس XDW A,B
هون يعني كل حرف رح ياخد حريعين
لانه DW

Scanned by CamScanner

Scanned by CamScanner

Ex:- Y DB 10H, 'A', 1111B

Sol:-

y	10H	y[0]
y+1	41H	y[1]
y+2	0FH	y[2]

1111B (لازم نحولها)

0000 1111

0 F ⇒ 0FH

ملاحظة:- يجب ان تكون القيم المخزنة في الـ Memory بالـ Hexa

~

Ex:- X DW 111B, 255

Sol

00
07H
00
FF

111B (حولها Hexa) → 0000 0000 0000 0111 ⇒ 07H

(0 0 0 7)

255 (حولها B) → 0000 0000 1111 1111 ⇒ FFH

(0 0 F F) (حولها Hexa)

~

Ex:- X DB 'hello'

Sol

X DB 'h', 'e', 'L', 'L', 'o' ≡ X DB 'hello'

'h'
'e'
'L'
'L'
'o'

Scanned by CamScanner

Scanned by CamScanner

* Instruction :- ⊗ مع نتعرف على أهم التعليمات :-

1] Mov Dest , Src
Reg Mem. Reg Mem immediate

هذه التعليمة تقوم بتخزين القيم في المتغيرات
يعني بضع المتغير (Memory. Reg.) في Dest وبضع حالات القيم (Reg. mem. imm.) في Src

Ex:- ملاحظة :-

Mov AL, 4 ① تعليمة Mov لا تؤثر بال Flag
Reg. immediate
AL=4 هيك هلأ

② يجب ان يكون ال Dest و ال Src متوافقين في الحجم ، 8bit و 8bit
Dest Src

③ ال Dest ممنوع يكون Immediate Value يعني Constant
Ex:- Mov 6, X → Error.

④ ممنوع ال Dest و ال Src يكون Memory location
Mov X, Y → Error. يعني .

* Errors :-

1] Mov AL, BX or Mov AL, 400H ⇒ ما بصير يكونوا diff. size يعني mov byte, word
2] Mov Ip, 120H ⇒ ما بقدر أخزن قيمة بال Cs وال Ip
3] Mov X, Y ⇒ ما بصير يكونوا كلهم Mem. Location او mov [Bx], [Si]
4] Mov 6, AL ⇒ ما بصير ال Dest يكون immediate
5] Mov Ds, Es ⇒ ما بصير يكونوا ال Dest و ال Src كلهم Seg. Reg

Scanned by CamScanner

Scanned by CamScanner

[6] MOV Ds, 1002H

Correction :- التصحيح

- لا يجوز تخزين ارقام مباشرة في (Ds) data seg.
- نقوم بالاستعانة بـ (Dx) Reg ثم ←

```
mov Dx, 1002H
mov Ds, Dx.
```

Example :- Write an assembly code to store the number from one to four in the memory starting at the address 800H : 300H

Sol :-

```
mov Dx, 800H
mov Ds, Dx
mov [300H], 1
mov [301H], 2
mov [302H], 3
mov [303H], 4
```

* هنا ان البداية 800H
ونعطي الـ offset ← 300H

Scanned by CamScanner

Scanned by CamScanner

* عمل الـ Addresing :-

[1] Regester Addressing Mode :-

Ex :- 1- Mov Ax, Bx
2- Mov AH, DL
3- Mov DH, BH

في هذه الحالة يكون التعامل مع Reg.

[2] Immediate Addressing Mode :-

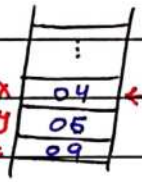
Ex :- 1- Mov AL, 40H (src)
2- Mov AH, 4H (src)

هون بتكون القيم بشكل مباشر في الـ Src

* تذكر :-
Mov 4, AL
Mov AL, 400H
} Error

[3] Indirect Addressing Mode :-

Ex :- X DB 04H
Mov Si, offset X
Mov CH, [Si]



رح يخزن القيمة اللي بتأشر عليها الـ Si في CH

وعشان استخدمنا الـ pointer سميناها Indirect.

* هون بخزن قيم من الـ Memory في متغيرات بالاستعانة بالـ pointer

[4] Direct Addressing Mode :-

Ex :- X DB 10H
Mov AH, [01050]



هون لهم نقناع إن pointer بأخذ القيم و تخزينها في AH

هون بخزن قيم في متغيرات عن طريق تحديد موقع القيمة في الـ Memory

ملاحظة :- [ ] :- تعني المحتوى الذي يوجد داخل القيمة المكوبوكة بين [ ]

Scanned by CamScanner
Scanned by CamScanner

[5] Direct Index Addressing Mode:-

* هنا نستخدم Si و Di فقط.

Ex:- X DB 01H, 02H, 03H, 04H

Mov Si, 1

Mov AL, X[Si]

لما يكون عندي قيمة لـ Si أو Di او اجمع لـ Si او Di قيمة فبتكون هاي العملية index.

روح على X Memory وأعمل shift بمقدار Si وخزن هاي القيمة في AL

X[0]	01	X
X[1]	02	X+1
X[2]	03	X+2
X[3]	04	X+3

=> AL = 02H

Mov AL, X[Si+1] ← هاي نفس Mov AL, X[Si] حتى لو ما كان محطي Si قيمة فتعتبر index.

[6] Base Relative Addressing Mode:-

* هنا نستخدم Bx و Bp فقط

Ex:- Mov AL, [Bx+1]

AL = 05H

Bx → 04H	
05H	

ولاحظة:-

الـ Bx ممكن يعتبر Reg و Pointer

[7] Base Index Addressing Mode:-

Bx BP Si Di

هون بتكون عندي العملية مختلطة Base مع index

Ex:- Mov Si, 2

Mov Cl, [Bx+Si] ; CL = 06H

Mov CH, [Bx+Si+2] ; CH = 08H

04	Bx = 0100H
05	
06	
07	0103H
08	0104H

* ولاحظات:-

Mov DL, [Bx][Si] ≡ Mov DL, [Bx+Si]

Mov DL, [Bx][Si] ≡ Mov DL, [Bx]+[Si]

* طبيعة الأسئلة على هذا الموضوع:-

1- بحيلك مثلاً اكتب مثال على Base index او نوع ثاني

2- تتبع

Scanned by CamScanner

Scanned by CamScanner

[2] Xchange (Xchg) :-

Xchg Dest , Src
(Dest: Reg, mem — Src: Reg, mem)

تستخدم هذه التعليمة لتبديل قيم بين متغيرين

Ex:-

```
X DB 4
Y DW 100H
Z DB 'A'
Mov AH, X    ; AH = 04H
Mov AL, y+1  ; AL = 01H
Xchg AL, AH  ; AH = 01H, AL = 04H
Mov CL, Z    ; CL = 41H
Xchg AL, CL  ; AL = 41H , CL = 04H
```

بالسؤال بده اتبع البرنامج وأوجد قيم كل من AL , AH , CL

عكس بينهم

عكس بينهم

AL = ? AH = ? CL = ?

⇒ AL = 41H AH = 01H CL = 04H

Ex:- Eqv → Xchg.

```
Mov AL, 04H
Mov DH, 07H
Mov BL, AL
Mov AL, DH
Mov DH, BL
```

نعبر اثنين بـ

≡ Xchg AL, DH

Scanned by CamScanner

Scanned by CamScanner

[3] LEA (Load Effective Add) :-

بستفيد منها اني بخلي الـ Pointer
LEA ≡ Offset
تؤشر على قيم Array في الـ Memory (Di , Si)

Ex:- Mov Si, offset x
LEA Bx, y
Mov Bx, offset y

⊛ EQU (Equivalent)

بتشبه مبدأ عمل الـ Constant في C++
وهي قيمة هوائية لا تخزن في الـ Memory

- C++ :- Const int X = 10;

- Assembly :- X EQU 10

⊛ Dup

تعليمة تستخدم لحجز عدد من الخانات في الـ memory

Ex:- X DB 4 Dup(?)
يعني: يكون حجزة 4 مواقع في الـ memory

ملاحظة:
اذا بدي احجز موقع واحد، ما في داعي استخدم Dup

Ex:- X DB 4, 2, ?, 5
يعني: يكون حجزة موقع واحد

Scanned by CamScanner

Scanned by CamScanner

* تنبيهات و أمور مهمة :-

① لغة الـ Assembly لا يهم اذا كان الحرف Capital او Small في الكلمات المحجوزة

② يتم تخزين جميع القيم في الـ Memory بنظام الـ Hexa

③ تحتوي الـ Memory على عدة خانات ، كل خانة عبارة عن 8bit

④ يتم قراءة و تخزين الرقم في الـ Memory من اليمين الى اليسار (6600)

00
66

⑤ O: Octal ، H: Hexa ، B: Binary ، D: Decimal ← اذا ما كان محطوط جنب الرقم اي دلالة يعتبر ايضاً Decimal

⑥ اي قيم تضع بين (' ') يمكن فصلها
مثلاً :- X DB 'ABC' ≡ X DB 'A','B','C'

⑦ أحرف الـ ascii code تعبر عن byte للحرف الواحد

⑧ يوجد اختلاف بين
- Mov AL, 245 ← هنا الرقم بنظام D فلما أحوله لـ Hexa يصير F5 H ← 4bit 4bit ← مقبول.
- Mov AL, 245 H ← هنا الرقم بنظام Hexa ، و عند Error الحجم مختلف

Scanned by CamScanner

Scanned by CamScanner

* اسئلة مراجعة :-

[1] Give an instruction an each the following Add. Mode :-

① Direct addressing mode :- Mov AL, [01015 H]

② Based relative addressing mode :- Mov CL, [Bx + 1]

③ Based index addressing mode :- Mov AH, [Bx + Si]

④ Indirect addressing mode :- Mov AL, X[Si]

[2]

① For Mov DI, [Si + 1] segment offset

Find :- ⓐ logical Address :- 1000 H : 1 H

ⓑ physical Address :- 10001 H

physical Address = 1000 H * 10H + 01 H = 10001 H

② The value of DI after executing the instruction

DI : 04 H

sol:- mov DI, [Si + 1]

mov DI, 04 H

③ The value of DL after

mov DL, [DI + 1]

sol:- Mov DL, [4 + 1]

Mov DL, 66 H

DL = 66 H

Si →	10000	03H
	10001	04H
	10002	00H
	10003	06H
	10004	10H
	10005	46H
	—	—
	1000E	5FH
	1000F	60H
Di →	10010	61H
	10011	62H
	10012	63H
	10013	64H
	10014	65H
	10015	66H

Scanned by CamScanner

Scanned by CamScanner

[3] If the contents of registers are as follow: DS=2003H, Si=1008H Di=1048H, BX=1008H, IP=1012H.
Using indexed addressingmode, write a code to load Dx register with the word contents of main memory location 210AAH.
(Use two lines of code or less.

Solution:-

السؤال معطيني Ds = 2003H ⇐ logical Reg
وبكيلي جد موقع 210AAH في الـ memory باستخدام الـ index

① Ds لا phy. add. بوجد الـ

phy = 2003H * 10H + 1048H
= 21038H

② و بعدها بوجد مقدار الـ offset حتى اوصل لـ 210AAH

210AA H
21038 H
072 H

⇒ Mov Dx, [Si+72H]

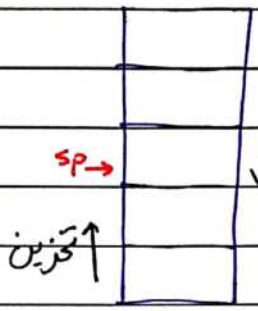
Scanned by CamScanner

Scanned by CamScanner

* Stack Instruction (Push, Pop)

- stack :-

هي جزء من الذاكرة (memory) حيث يكون التخزين فيها مؤقت ويتم التخزين فيها من الأسفل إلى الأعلى والاسترجاع من الأعلى إلى الأسفل.



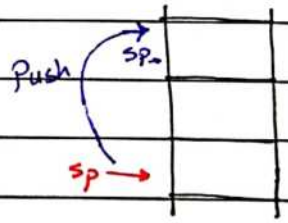
- Push :-

Push Src ← لازم يكون حجم الـ src 16bit

Reg mem imm.

تقوم هذه التعليمة بتخزين القيم في الـ stack حيث تقل على تحويل المؤشر SP إلى الأعلى بمقدار مرتين بحيث أن الـ Push عبارة عن 16bit والتخزين في الـ stack عبارة عن 8 bit

* لما أعمل Push :- يحرك المؤشر إلى أعلى مرتين بخزن القيمة (ينقل قيمة Src إلى stack)



Ex:- X DW 1122 H

Mov Dx, 4455H

Push 6677 H

Push X

الحل :-



* ملاحظة :-
Push AL
Error
16bit فقط

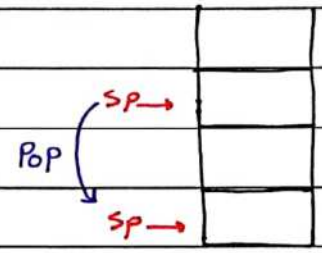
Scanned by CamScanner
Scanned by CamScanner

- Pop :-

Pop Src ← لازم يكون حجمه 16bit
Reg mem

تقوم هذه التعليمة بسحب القيم من ال Stack حيث تقل عن سحب القيمة ثم تحرير المؤشر Sp إلى أسفل بمقدار مربعين كونه حجمها 16 bit

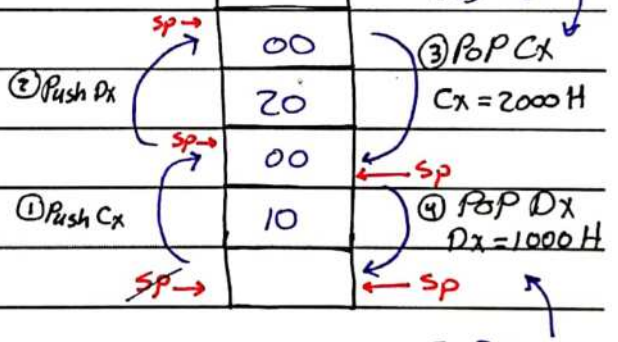
تقل ال Pop بسحب القيمة وتخزينها في ال Src ثم تحرك المؤشر إلى أسفل.



Ex:- equ Xchg Using Stack
Xchg Dx, Cx

Sol:-

```
Mov Cx, 1000H
Mov Dx, 2000H
Push Cx
Push Dx
Pop Cx ; Cx = 2000H
Pop Dx ; Dx = 1000H
```



هون اخذ قيمة Cx بعدين نزل.

هون اخذ قيمة ال Dx بعدين نزل.

Scanned by CamScanner

Scanned by CamScanner

[4] ADD :-

ADD Dest , Src
Reg mem Reg mem imm.

هذه التعليمة تقوم بعملية جمع بين متغيرين
حيث تقوم بجمع وتخزين ناتج الجمع في المتغير الاول الموجود في الـ Dest

⇒ Dest = Dest + Src

ملاحظة :-
الـ Add تؤثر على الـ Flags في جميع الـ

ملاحظة :-
(و) :- هذه العبارة تعني تعليمة اي لا تؤثر على البرنامج

Ex :- CH اجمع رقمين وخزن الناتج في
Sol:- Mov AL, 30H
Mov CH, 40H
ADD CH, AL ; CH = 70H
* لو كان ADD AL, CH ⇐ كان AL = 70H

Ex :- X DB 'A'
y DB ?
ADD X, 2 , X = 41 + 2 = 43H
Mov DH, X , DH = 43H
Mov y, DH , y = 43H
y = ? ⇒ y = 43H.

تذكر :-
Mov x, y ⟶ Error

Scanned by CamScanner

Scanned by CamScanner

[5] INC :-

INC Dest
Reg mem

INC AL ≡ ADD AL,1

تقوم هذه التعليمة بزيادة مقدار one للمتغير
وتؤثر على جميع الـ Flag ماعدا CF

Ex:- MOV Ax, 0FEH
INC Ax
Ax = ?? ⇒ Ax = 00FF

Ex:- X DB 1,2,3,4
Y DB 5
ADD X,2 ; X=1 → X[0]=3
ADD X+2, 3 ; X[2]=3 → X[2]=6
INC X ; X[0]=3 $\xrightarrow{inc}$ X[0]=4
INC X[3] ; X[3]=4 $\xrightarrow{inc}$ X[3]=5
INC X+4 ; X[4]=Y[0] $\xrightarrow{inc}$ Y[0]=06H

X[0]	~~1~~ 03H 04H	
X[1]	2	
X[2]	~~3~~ 06H	
X[3]	~~4~~ 05H	
X[4]	~~5~~ 06H	Y[0]

نلاحظ هنا انه لو انتهت عناصر X في الـ memory فإنه ينتقل إلى العناصر الأخرى المخزنة في الـ mem لحد

[6] ADC :-

ADC Dest, Src
Reg mem Reg mem imm

هذه التعليمة تقوم بعملية الجمع بين متغيرين مع اضافة قيمة الـ CF للناتج

Dest = Dest + Src + CF

Ex:- BX AX
4411 FF00

32bit باعتبار البدء الجمع بقسمين

DX → 1322 1011 ← + CX

فيوزع الرقمين على Reg حتى اسهل على المعالج

5734 0F11
Bx Ax

ADD Ax,Cx
كون بداية العملية الحسابية

ADC
كون يجب اخذ الناتج Carry من

Scanned by CamScanner

Scanned by CamScanner

[7] SUB :-

SUB Dest , Src
(Dest: Reg, mem ; Src: Reg, mem, imm)

هذه التعليمة تقوم بطرح بين متغيرين وتخزن قيمة الناتج في الـ Dest

Dest = Dest - Src

* وهذه التعليمة تؤثر على جميع الـ Flags

Ex:- تتبع

Mov AL , ϕEE H — 1101 1101
Mov AH , 2C H — 0010 1100 −
SUB AL , AH — 1011 0001
(1011 = B , 0001 = 1)

⇒ AL = ??

⇒ AL = ϕB1 H

[8] DEC :- تكافؤ

DEC Dest ≡ SUB AL , 1
(Dest: Reg, mem ; Dest)

هذه التعليمة تقوم بطرح مقدار one من المتغير وتخزن الناتج في الـ Dest

Ex:- Mov AL , 07H

DEC AL

⇒ AL = ?? ⇒ AL = 06H

[9] SBB :-

SBB Dest , Src
(Dest: Reg, mem ; Src: Reg, mem, imm)

هذه التعليمة تقوم بطرح بين متغيرين وطرح معهم قيمة الـ CF

⇒ Dest = Dest - Src - CF

Scanned by CamScanner

Scanned by CamScanner

Ex:- 0033 3567 (Bx, Ax)

Dx ← 0021 4100 → Cx

0011 F467

Bx = 0011 — Ax →

هون لو استخدمت sub

لأنه بداية العملية العسابية بطلع الجواب غلط لأنه ممكن يكون عندي CF من العملية السابقة

SBB Bx, Dx

Bx = 0011 H

هون استخدمت sub

لأنه CF يوجد

sub Ax, Cx

Ax = F467 H

10 Compare (cmp)

هذه التعليمة تقوم بعملية الطرح دون ان تخزن الناتج في الـ Dest

Cmp Dest, Src

Reg mem Reg mem imm

SF = 0 → Dest ≥ Src

SF = 1 → Dest ≤ Src

ZF = 1 → Dest = Src

ZF = 0 → Dest ≠ Src

ملاحظة:-

① لا تغير على قيمة الـ Dest

② تؤثر على الـ Flags ، بالأخص:-

SF / CF / ZF / OF

Dest − Src → فقط

أي بدون تخزين

Scanned by CamScanner

Scanned by CamScanner

Ex :- مثال

```
Mov Si, offset x
Mov Di, offset y
Mov Ax, [Si]   ; Ax = 4567H
Mov Bx, [Si+2] ; Bx = 0033H
Mov Dx, [Di]   ; Dx = 3100H
Mov Cx, [Di+2] ; Cx = 0021H
ADD Ax, Dx ; Ax = Ax + Dx => Ax = 7667H → CF=0
ADC Bx, Cx ; Bx + Cx + CF => Bx = 0054H
Mov Si, offset z
Mov [Si], Ax
Mov [Si+2], Bx
```

Si →	67	x
	45	
	33	
	00	
Di →	00	y
	31	
	21	
	00	
	67	z
	76	
	54	
	00	

```
x DD 334567H
y DD 213100H
z DD ?        => z = 00547667
```


Scanned by CamScanner

Scanned by CamScanner

11 Jump if Carry :- (JC)

هذه التعليمة يعني انه انتقل
الى عملية العمل (L) اذا كان CF=1
غير ذلك ، لا تنتقل.

Ex:- -----

L: mov
JC L → (L) هذه تعني ارجع الى عملية
اذا كان CF=1 ، غير ذلك لا ترجع

12 Mull :-

Mull Src
Reg mem
ينقسم الى
- Unsigned Mull → 8bit → Ax = AL * Src
- Signed IMull → 16 bit → Dx.Ax = Ax * Src
  (Dx: high, Ax: Low)

هي تعليمة تقوم بعملية ضرب بين المتغيرات ، وتنقسم الارقام إلى :-
1. IMull نستخدم معها Signed number
2. Mull نستخدم معها Unsigned number

وتقسم هذه التعليمة كما الـ AL في حالة الـ 8bit والـ Ax في حالة الـ 16 bit
فمثلاً Mull Src ← قيمة الـ Src بنضربها بقيمة الـ AL او الـ Ax حسب الحجم إي عندي

ملاحظة:- هذه التعليمة تؤثر على OF / CF

Scanned by CamScanner

Scanned by CamScanner

- Unsigned Numbers (Mull)

الـ Mull ما بنتعامل مع الاشارات
يعني لازم الرقم يكون موجب.
في حالة كان عندي الرقم سالب وعندي Mull
بعمل للرقم 2nd comp. وبعدين بعربه

8 bit → Ax = AL * Src

16 bit → Dx, Ax = Ax * Src
(High 16bit) (Low 16 bit)

Ex :- Mov CL, 04H
Mov AL, 02H
Mull CL ; AL * CL Ax = AL * Src
2 * 4 = 08H Ax = 08H.

Ex :- Mov DH, -2 الرقم صالب → 2nd comp → 0000 0010 → 2nd comp → 1111 1110 (F E)
Mov AL, 1
Mull DH ; Ax = 1 * 00FEH
Ax = 00FEH.

Ex :- Mov Ax, -2 ; -2 ≡ 0000 0000 0000 0010 → 2nd comp → 1111 1111 1111 1110 (F F F E)
Mov Cx, 1
Mull CX ; Dx.Ax = Ax * Src
= FFFE * 1 Ax = FFFE H
Ax = FFFE

ليش بنا تيجي تخزن في الـ Ax ؟
لانه عندي 16 bit وهون بتخزن النا تيج في Dx.Ax هيك:-
Ax ← Low جزء 16 bit
Dx ← High جزء 16 bit وبنا تيج عندي (16bit) يعني بتخزن في Low اي في Ax
واذا طلع زيادة عن 16bit بتخزن بالجوان في Dx.Ax

Scanned by CamScanner

Scanned by CamScanner

- Signed Numbers (IMULL)

8 bit → Ax = AL * Src

16 bit → Dx . Ax = Ax * Src
(Dx: high, Ax: low)

IMULL تتعامل مع الاشارة يعني يكون مقبول عندي الرقم السالب

Ex:- Mov DH , -2
Mov AL , 1
IMULL DH ; Ax = 1 * -2

* هون عندي مقبول الرقم السالب، بس الجواب بدو 2nd comp.

Ax = -2 → Ax = 0000 0000 0000 0010
Ax = FFFEH → 1111 1111 1111 1110 (2nd comp)
F F F E

* خلينا ناخد مثال نشوف الفرق بين الـ IMull والـ Mull

Ex:- Mov DL , ØFFH
Mov AL , 2H
Mull DL
Ax = FF H * 2H
Ax = 255 * 2
Ax = 510
Ax = 01FE H.

1111 1111
F F
mull 255

Ex:- Mov DL , ØFF H
MovAL , 2H
IMull DL
Ax = 2 * -1
Ax = -2 (2nd comp.)
Ax = FFFEH
imull 0000 0001
-1

ملاحظة:-
- الـ Mull بتتعامل مع رقم موجب بس
- الـ IMull بتتعامل مع رقم سالب وموجب

ملاحظة:- لو كان عندي Mull AL
⇒ Ax = AL * AL

Scanned by CamScanner

Scanned by CamScanner

13 Div :-

Div Src
Reg mem

تقسيم إلى
- unsigned (Div)
- Signed (IDiv)

هذه التعليمة تقوم بقسمة متغيرين
حيث تخزن الباقي في AH
والناتج في AL

AH	AL
Rem.	Result

8bit → Ax = Ax / Src
AH الباقي — AL الناتج

حيث تؤثر كل CF و OF فقط.

16 bit → DX.Ax = DX.Ax / Src
rem. Result.

* في حالة:-
① 8bit يخزن: الناتج ← AL ، الباقي ← AH

② 16bit يخزن: الناتج ← AX ، الباقي ← DX

Result
....

Rem. ← إشارة الباقي نفس إشارة القاسم

ملاحظة:-
في Div و IDiv تعامل دائماً مع الـ Ax لذلك إذا عشر AL في AH فيعتبرهم سالب بعمل 2nd comp عشرة.

Div	IDiv
لا نهتم للإشارة بحيث إذا كان آخر bit (1) ما بهتم وبقسم عادي	نهتم للإشارة بحيث إذا كان آخر bit (1) بعمل 2nd comp للرقم بعدين بنقسم.

Scanned by CamScanner

Scanned by CamScanner

Ex:- Mov Ax, 08H
Mov CL, 02H
Div CL

Sol:-
16 bit → Ax = Ax / CL
= 08H / 02H
Ax = 0004 H
Ax = [00 | 04]
AH AL

4 → AL
2 | 8
8
0
↓ AH

⊛ Mov AH, 08H → Ax = [08 | 00] (AH AL)
⊛ Mov AL, 08H → Ax = [00 | 08] (AH AL)

Ex:- Mov Ax, -8
Mov BH, 2
IDiv BH

Sol:-
2 | -8
-8
0

Ax = [00 | FC] (AH AL) ← 2nd comp. -4
Rem result

Ex:- Mov BL, -2
Mov AH, -4
IDiv BL

Sol:-
AH = -4 → 2nd comp.
AH = FC
Ax = [FC | 00]
Ax = 1111 1100 0000 0000
Ax = 0000 0100 0000 0000
2nd comp.
Ax = 1024
AL = 512 D
AH = 00

AL
-2 | -1024
AH.

Ex:- Mov Bx, -2
Mov AL, -4
IDiv Bx

Sol:-
AL = -4 → 2nd comp. 0000 0100
AL = FCH
Ax = [00 | FC]
Ax = 0000 0000 1111 1100

-AL
-2 | +252
+AH

Scanned by CamScanner

Scanned by CamScanner

CTI :- Control Transfer Instruction

Unconditional
- Call
- JMP
- INT
- RET

Conditional.
- Flag.
  - JZ
  - JNZ
  - JO
  - JNO
  - JC
  - JNC
  - JP
  - JNP
  - JS
  - JNS
- Relational
  - Unsigned.
    - JA
    - JNA
    - JAE
    - JB
    - JNB
    - JBE
    - JE
    - JNE
  - Signed
    - JG
    - JNG
    - JGE
    - JL
    - JNL
    - JLE
    - JE
    - JNE

JZ → if ZF = 1 → go to Label
JNZ → if ZF = 0 → go to label
JC → if CF = 1 → go to label
JNC → if CF = 0 → go to label
JS → if SF = 1 → go to label
JNS → if SF = 0 → go to label
JO → if OF = 1 → go to label
JNO → if OF = 0 → go to label
JP → if PF = 1 → go to label
JNP → if PF = 0 → go to label

Scanned by CamScanner

Scanned by CamScanner

14 Jump Label :-

تستخدم هذه التعليمة كأداة شرط ، حيث يتم وضعها بالعادة بعد Cmp وتحتوي على شرط ، في حال كان الشرط صحيحاً تقفز إلى تعليمة معينة ، أما اذا كان خاطئ تنفذ التعليمات التي تليها .

Type of Jump :-

JE :- تنفذ الـ Lable في حال ZF=1

JC :- تنفذ الـ Lable اذا كان CF=1

JA :- تنفذ الـ Lable اذا كان المتغير الاول اكبر من الثاني (لا ننظر إلى الاشارة) .

JG :- تنفذ الـ Lable اذا كان المتغير الاول اكبر من الثاني (مراعاة الاشارة) .

JB :- تنفذ الـ Lable اذا كان المتغير الاول اصغر من الثاني (لا ننظر إلى الاشارة) .

JL :- تنفذ الـ Lable اذا كان المتغير الاول اصغر من الثاني (مع مراعاة الاشارة) .

JZ :- تنفذ الـ Lable اذا كان ZF=1

JNE :- تنفذ الـ Lable اذا كان ZF=0

JNC :- تنفذ الـ Lable اذا كان CF=0

JNZ :- تنفذ الـ Lable اذا كان ZF=0

JBE :- تنفذ الـ Lable اذا كان اقل من أو يساوي

JCXZ :- تنفذ الـ Lable اذا كان Cx=0

JAF :- تنفذ الـ Lable اذا كان اكبر من أو يساوي

Scanned by CamScanner

Scanned by CamScanner

Adel Albasheer

* تحويل برامج من C++ إلى لغة Assembly :-

[1] if (x>5){
y++;
z++; }
else
y--;

Solution :-

```
X DB 4
Y DB 3
Z DB 2
Cmp X,5
JB L1
INC y
INC Z
Jmp exit
L1 : Dec y
exit : HLT
```

[2] For(int i=0; i<=5; i++)
{ x=y+5;
z=x;
}

Solution :-

```
org 100 H
Mov Cx,0
L1 : Mov AL,y.
ADD AL,5
Mov x,AL
Mov BL,X
Mov Z,BL
INC CX
Cmp Cx,5
JBE L1
HLT
X DB 0
y DB 5
z DB ?
```


Scanned by CamScanner

Scanned by CamScanner

* Logical instruction :-

① AND :-

Dest , Src

Reg. mem. Reg. mem. imm

Dest = Dest. AND Src

ملاحظات :-
① الـ AND تؤثر على الـ Dest. فقط
② ممكن استخدام AND في معرفة ان الرقم (even/odd) او (+/-)

Example :-

mov AH, 77H
mov BL, 33H
AND AH, BL

Sol :→

```
  0111 0111   ← 77H
& 0011 0011   ← 33H
-------------
  0011 0011
```

⇒ AH = 00110011 = 33H
BL = 33H.

② OR :-

ملاحظة :-
الـ OR تؤثر على الـ Dest. فقط.

Dest , Src

Reg. mem. Reg. mem. imm.

Dest = Dest. OR Src

Example :-

mov CL, ØFBH
mov DH, ØEH
OR CL, DH

Sol :-

```
  1111 1011   ← ØFBH
| 0000 1110   ← ØEH
-------------
  1111 1111
```

⇒ CL = 11111111 = ØFFH
DH = ØEH

Scanned by CamScanner

Scanned by CamScanner

③ XOR :-

Dest , Src

Reg. mem. Reg. mem. imm

ملاحظة :- الـ XOR تؤثر على الـ Dest. فقط .

$1 \oplus 0 = 1$
$1 \oplus 1 = 0$
$0 \oplus 1 = 1$
$0 \oplus 0 = 0$

zero = تشابه
1 = اختلاف

① mov Ax,0
② and Ax,0
③ Sub Ax,x
= XOR Ax,Ax

جميعهم بعمله نفس النتيجة
بس الـ mov افضلهم لانها لا تؤثر على الـ Flags

Example:-

mov CH, 77H
mov DH, 30H
XOR DH, CH

sol:→
0111 0111 ← 77H
0011 0000 ← 30H
1100 0111
⇒ DH = 0100 0111 = 47H
CH = 30H.

④ NOT

ملاحظة :- الـ Not تؤثر على الـ Dest فقط .

Dest

Dest = !Dest.

Not → First Comp.

Example:-

mov cl, 1111 B
Not cl

sol→
!CL = ! 0000 1111
⇒ CL = 1111 0000 = ϕF0 H.

Scanned by CamScanner

Scanned by CamScanner

⑤ NEG

Dest.

Dest = !Dest

Neg = 2nd comp.

ملاحظة:- الـ Neg تؤثر على الـ Dest فقط.

Example:-

mov AL, -2

Neg AL 2nd comp 2 ≡ 00000010

Neg AL 2nd comp -2 ≡ 11111110

⑥ Test

Dest., Src

* نفس مبدأ عمل الـ AND لكن
X تغير على قيمة الـ Dest.

Dest ≠ Dest. test Src

* تؤثر على:-

① ZF
② PF
③ SF

Scanned by CamScanner

Scanned by CamScanner

* Convert from Small to Cabital. ('a' → 'A')

Solution:-

61 H = 'a' → 0110 0001

تحويل ↓ 0100 1111 and.

41 H = 'A' → 0100 0001

mov AH, 'a'
AND AH, ϕ DF H
⇒ AH = 'A' = 41 H.

* Convert from Capital to Small ('A' → 'a')

Solution:-

41 H = 'A' → 0100 0001

تحويل ↓ 0010 0000 OR

61 H = 'a' → 0110 0001

mov CH, 'A'
OR CH, 20H
⇒ CH = 'a' = 61 H.

Scanned by CamScanner

Scanned by CamScanner

* Set & Reset.

Set → 1 Reset → 0
↳ (clear)

Example:- Set bit #3 in AL = ϕ F4

Sol:-

```
  7 6 5 4 3 2 1 0
  1 1 1 1 0 1 0 0      OR AL, 8H
  0 0 0 0 1 0 0 0  OR
  -----------------
  1 1 1 1 1 1 0 0
```

* Bit manupiadion instruction:-

#Shift:-

diff. size ← يسمح

SHR Dest, Shift #No. (1) أكبر من

Dest: mem. / Reg.
Shift: 1 / CL

* يمكن استخدامه لتحديد ال odd و ال even (CF)

```
1 1 0 0 1 1 0 0   shr(1) →   0 1 1 0 0 1 1 0   CF = 0
        ↓ CF

0 1 1 0 0 1 1 0   shr →      0 0 1 1 0 0 1 1   CF = 0
        ↓ CF

0 0 1 1 0 0 1 1   shr →      0 0 0 1 1 0 0 1   CF = 1
        ↓ CF
```


Scanned by CamScanner

Scanned by CamScanner

Example:-
mov AL, 11001100B
mov CL, 2
Shr AL, CL
Sol:-
11001100 → 00110011
CF=0

Example:-
mov BX, ØFFFF
mov CL, 8
Shr BX, CL
Sol:-
1111 1111 1111 1111 → 00FFH
0 0 F F
CF=1

[2] ShL Dest, Shift # (No.)
mem Reg — 1 CL (أكثر من 1)

Posit or Neg. يمكن استخدام الـ CF لمعرفة

11001100 —ShL→ 10011000 CF=1
CF

11001000 —ShL→ 10010000 CF=1
CF

01010000 —ShL→ 10100000 CF=0
CF=0

Example:-
mov DH, 11001100B
① ShL DH, 1
① ShL DH, 1
= mov CL, 2
ShL DH, CL

Sol:- ① 11001100 —ShL→ 10011000 → CF=1
CF
② 10011000 —ShL→ 00110000 → CF=1
CF

Scanned by CamScanner

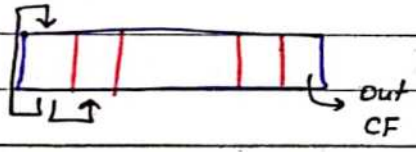
3 SAr Dest, Shift #No

Reg mem — 1 CL

* هنا يسمح باستخدام Ax, AL مثلاً diff. size

* بتعتمد SF اذا الرقم سالب بدخل (1)

بتعتمد SF اذا الرقم موجب بدخل (0)



1100110 SAr → 1110011 CF=0

1110011 CF → SAr 1111001 CF=1

Example:-

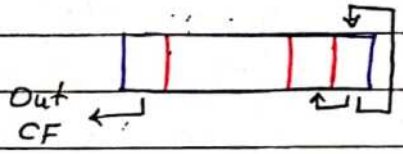
mov Bx, 8000H Sol:- 1000 0000 0000 0000

mov CL, 16 1111 1111 1111 1111 بدخل one 16

SAr Bx, CL → Bx = FFFFH CF=1

4 SAL Dest, Shift #No

Reg mem. — 1 CL



1100110 SAL → 1001100 CF=1

CF

Scanned by CamScanner

Scanned by CamScanner

Rotate :-

diff. size → error

1] RoR Dest, rol #
mem Reg — 1 CL — No.

11001100 —RoR→ 01100110 CF = 0
CF

01100110 —RoR→ 00110011 CF = 0
CF

00110011 —RoR→ 10011001 CF = 1
CF

2] RoL Dest, rol #
mem Reg. — 1 CL — No.

11001100 —RoL→ 10011001 CF = 1
CF

10011001 —RoL→ 00110011 CF = 1
CF

Example:-

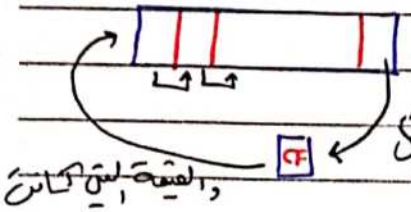
```
mov CH, 11001100
mov CL, 2
RoL CH, CL
```

sol → 00110011 = CH
1 = CF
33 02 = CX
CH CL

Scanned by CamScanner

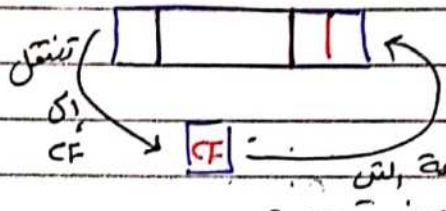
Scanned by CamScanner

[3] RCR Dest, Rol



والقيمة اللي كانت في الـ CF هي اللي تنتقل.

[4] RCL Dest, Rol



القيمة اللي كانت في الـ CF هي اللي تنتقل.

#Loop :-

like Jmp → Loop label.

1. CX = CX - 1
2. Jump to label if CX ≠ 0
3. Execution (تنفيذ)

الخروج من الشرط CX=0 اذا

Example:-

```
mov CX, 4
mov X, 0
L1: Add X, 1
    Dec CX
    Cmp CX, 0
    JNZ L1
```

≡

```
mov X, 0
mov CX, 4
L1: Add X, 1
    Loop L1
```


Scanned by CamScanner

Scanned by CamScanner

#LoopE Label:-

LoopE ≡ LoopZ

1. CX = CX − 1
2. CX ≠ 0 & if ZF = 1 (ZF=1 يعني الجواب صفر)
3. Jump to label

Example:- Find the different elements in array & stored in array Y

```
X DB 'a','a','a','a','b','a','c'
Y DB 2 Dup(?)
```

Sol:-

```
    mov cx,7
    mov Si,-1
L1:
    inc Si
    Cmp X[Si],'a'
    LoopE L1
    mov AL,X[Si]
    mov Y[Di],AL
    inc Di
    Cmp Si,7
    JNZ L1
    HLT
```

(inc تؤثر على جميع Flags عدا CF)

هاد تعود خاطئ كنه inc تؤثر على ZF يعني بعد Cmp ← ZF = ! وبعدها فايضل على inc يصير ZF=0 وبالتالي اخلى شرط LoopE

```
L1:- cmp X[Si],'a'
     inc Si
     loopE L1
```


Scanned by CamScanner

Scanned by CamScanner

LoopNE Label :-

LoopNE ≡ LoopNZ

1. Cx = Cx - 1
2. if Cx ≠ 0 & if ZF = 0 — ZF = 0 ، تعني ان الجواب ليس صفر
3. Jump to label.

Example:- X DB 'a','b','c','d','e','F','e'
if 'e' found, write F in DH Reg.

Sol:-

```
mov Si,-1
mov Cx,7
L1:
inc Si          ←  الـ inc قبل الـ Cmp
Cmp X[Si],'e'      حتى ما تؤثر على الـ ZF
loopne L1
JNZ L2
mov AH,'F'
mov X[Di],AH
inc Di
cmp Cx,0
Jne L1
L2: HLT
X DB 'a','b','c','d','e','f','e'
Y DB 2 Dup(?)
```

Answer:-
Y = 'f','f'

Scanned by CamScanner

Scanned by CamScanner

Example:- Write assembly code, shL all elements in array(x)

shift بمقدار واحد *

```
x DB 3,4,6,7
```

Sol:-

```
mov Si,0
mov Cx,4
L1:
SHL x[Si],1
inc Si
loop L1
HLT
```

x DB 3,4,6,7 —(بعد shift)→ x DB 6,8,12,14 (0CH 0EH)

* لاحظ ان عملية SHL هي عبارة عن عملية ضرب بمقدار (2).

Example:- Write assembly Code, shr all elements in array(x)

shift بمقدار واحد *

```
x DB 2,4,6,8
```

Sol:-

```
mov Si,0
mov Cx,4
L1:
SHR x[Si],1
inc Si
loop L1
HLT
```

(بعد shift)→ x DB 1,2,3,4

* لاحظ ان عملية SHR هي عبارة عن عملية قسمة على (2).

Scanned by CamScanner

Scanned by CamScanner

Procedures:-

*The syntax for procedure declaration:-

```
name PROC
    ; here goes the code
    ; of the procedure

    RET
name ENDP
```

Example:-

```
org 100H
call m1
mov Ax,2
RET ; HLT
m1 PROC
mov Bx,5
RET
m1 ENDP
```

استدعاء PROC الـ

انتهاء ثم يرجوع إلى ما بعد الـ Call

Example:-

```
org 100H
mov AL, 4
mov BL,2
call fact
HLT
Fact PROC
Add AL,BL
mov CL, AL
RET
Fact ENDP.
```


Scanned by CamScanner

Scanned by CamScanner

Convert from char. to Binary.

41 H = 'A' ⟶ 0100 0001

34 H = '4' ⟶ 0011 0100

Ex:- Convert array X & stored in Y = X+20

```
X DB '123'
Y DB ?
Z DB 100,10,1
```

Sol:-

```
mov Si, 0
mov Cx, 3
L1: mov Al, X[Si]   ; array حتى يمشي كل عناصر ال
And Al, 0FH         ; لتحويل اكثر رقم إلى 3,2,1
Mul Z, [Si]         ; حتى يتم تحويل اكثر رقم إلى مئات وعشرات وآحاد و
Add DL, AL          ; لتحوله إلى رقم 123
inc Si
Loop L1
mov Y, DL
Add Y, 20
HLT
```


Scanned by CamScanner

Scanned by CamScanner

Convert from Binary to char. to Binary.

Example:- Convert Array X

XDB 123

123 ⟶ '123' = 31,32,33

Sol:-

```
org 100H
mov AL, X
mov Cx, 3
L1: mov AH, 0
Div Y[DiJ
OR Al, 30
mov Z[DiJ, Al
mov AL, AH
inc Di
Loop L1
HLT
XDB 123
Y DB 100, 10, 1
Z DB 3 Dup(?)
```

Z = 31, 32, 33

Another Solution:-

```
mov Di, 0
mov Al, X
L1: mov AH, 0
Div Z[DiJ
OR Al, 30H
mov Y[DiJ, AL
inc Di
mov AL, AH
Cmp Di, 3
JB L1
HLT
X DB 123
Y DB 100, 10, 1
Z DB 3 Dup(?)
```

Y = 31, 32, 33

شرح بمثال ⟵

Scanned by CamScanner

Scanned by CamScanner

شرح بمثال لسابقه :-

123 ⟶ '123'

AL = 123
AH = 0
Y[1] = 100

1 → AL ①

100 | 123
100
23 → AH

* الآن نعدل قيمة AL إلى 31 = AL
عن طريق OR AL,30

* بعدين ننقل قيمة AH إلى AL ⟸ AH=AL=23

2 → AL ②

AL = 23
AH = 0
Y[2] = 10

10 | 23
20
3 → AH

* الآن نعدل قيمة AL إلى 32
عن طريق OR AL,30

* بعدين ننقل AH إلى AL ⟸ AH=AL=3

:
وهكذا!

Scanned by CamScanner

Scanned by CamScanner

*String :-

هي تعتبر اداة لنسخ من mem الى mem او من mem الى reg او من reg الى mem عن طريق مجموعة تعليمات..

* movsB	* movsW
Copy byte from	Copy words from
Si → Di (mem to mem)	Si → Di (mem to mem)

* ملاحظة:- يعتمد اعتماد كلي على الـ DF

DF		DF	
DF=0	DF=1	DF=0	DF=1
inc Si	Dec Si	add Si,2	Sub Si,2
inc Di	Dec Di	add Di,2	Sub Di,2

* ملاحظة:- بما اني بتعامل مع Si, Di ← two pointer يعني بدي two mem.

Example:- تتبع

```
mov Cx,5
mov Si,offset x
mov Di,offset y
mov AL,[Si]
mov [Di],AL
Inc Si
Inc Di
Loop L
```

≡ movsB

يمكن استبدال هذا الجزء من الكود بـ movsB

تعني: STD → DF = 1

تعني: CLD → DF = 0

اذا لم يتم التحديد باستخدام هذه الجمل، البرنامج يعتبرها DF=0 وهي default

تذكر:- mov [Si],[Di] → error

Scanned by CamScanner

Scanned by CamScanner

* نحل المثال السابق بكود الحالتين :-

① DF=0 اذا كان

CLD ; DF=0 أن

mov Cx,5

mov Si,offset x ; } رح تبلش من اليسار
mov Di,offset y ; } ويزيد بـ 4 (بمشي لليمين)

L1: movsB . DF=0 لأنها

Loop L1

② DF = 1 اذا كان

STD ; DF=1 أن

mov Cx,5

mov Si,offset x ; } رح تبلش من اليمين بـ 4
mov Di,offset y ; } Dec لأنه رح يقل

L1: movsB DF=1 بسبب

Loop L1

Scanned by CamScanner

Scanned by CamScanner

Example:- مثال على ④ movsw

mov Cx, 2 ; لما احتاج ابي ارجع لقارئ كلمة word

CLD ; DF=0 (2 word ينقل)

mov Si, offset x

mov Di, offset y

L: movsw ⟶ DF=0 ↓ add Si, 2 / add Di, 2

Loop L

HLT

x DB '1', '2', '3', '4' coby يدخل

y DB 4 Dup(?)

x	1	← Si ①
	2	
	3	← Si ③
	4	
y	1	← Di ②
	2	
	3	← Di ④
	4	

one word Without Loop & use movsD :- لو كلمة x

[2] LODS instruction:-

LODSB

Load byte at Si ⟶ Reg. (mem to AL)

LODSW

Load byte at Si ⟶ Ax (mem to Reg.)

نتعامل مع Si فقط

DF

DF=0 → inc Si

DF=1 → dec Si

DF

DF=0 → add Si, 2

DF=1 → Sub Si, 2

⑧ Copy byte/word from mem to reg (AL, Ax)

byte ← AL

word ← Ax

ينقل الكود العظيم ابي

Scanned by CamScanner

Scanned by CamScanner

Example:-

```
ORG 100H
mov Si, offset X      ; DF=0
mov Di, 0
CLD                   ; DF=0
mov Cx, 4
L:
LodSB                 ; mov AL, Si
mov Y[Di], AL         ; inc Si
Inc Di
Loop L
HLT
X DB 1, 2, 3, 4
Y DB 4 Dup(?)
```

لو كانت DF=1
بتعمل (mov Si, offset X+4)
وبتكمل البرنامج وتعمل mov Di, 4
وبتعمل تحت dec Di

لاحظ هنا انه غير
انه Inc Di بتعمل
التعليمة Lod
لا تؤثر عليها

* REP : Repeat

REP movSB
↓ Cx ≠ 0 يتحقق
↓ Cx = Cx - 1 يطرح
↓ Run. ينفذ

* Loop.

Loop L1
↓ Cx = Cx - 1 يطرح
↓ Cx ≠ 0 يتحقق
↓ Run. ينفذ

REP ≡ Loop
REPE ≡ LoopE
REPNE ≡ LoopNE

Scanned by CamScanner

Scanned by CamScanner

[3]

* STOSB
Store byte at
AL ⟶ Di
Reg. to mem

تتعامل مع Di فقط

* STOSW
store word at
Ax ⟶ Di
Reg. to mem.

DF
DF=0: Inc Di
DF=1: Dec Di

DF
DF=0: Add Di,2
DF=1: Sub Di,2

Example:- تتبع

```
mov Si,offset x
mov Di, offset y.
CLD
mov AL, '*'
STOSB   تتعامل مع Di فقط و
LodSB   تتعامل مع Si فقط و
HLT
y DB ?
x DB 0
```

0100 H	'*'	y ← Di
0101 H	0	x ← Si ← Di
0102 H		← Si

هنا كان عندي AL='*' عند الوصول
إلى التعليمة STOSB سوف تنتقل
قيمة AL إلى memory إلى y[Di]
وتزيد قيمة Di واحد فيصبح Di و Si يؤشران
على نفس الموقع، وعند الوصول إلى التعليمة LodSB
سوف تنتقل قيمة الـ memory إلى x[Si] إلى AL Reg.
فيصبح لدي:-
AL=0 y=0 Si=0102H
x='*' Di=0101H

Scanned by CamScanner

Example:-

```
mov Cx, 4
mov Si, offset x
mov Di, offset y.
L1:
LODSB   ] = MOVSB
STOSB
Loop L1
HLT
x DB 'ABCD'
y DB 4 Dup(?)
```

هيك البرنامج
يقوم بنسخ قيمة
x إلى y

Scanned by CamScanner

Scanned by CamScanner

[4] Scas Instruction.

* ScasB

Cmp AL, [Di]

DF = 0	DF = 1
CLD	STD
inc Di	dec Di

* ScasW

Cmp Ax, [Di]

DF = 0	DF = 1
CLD	STD
Add Di, 2	Sub Di, 2

* Compare (AL / Ax) with Di
* Update Di

تستخدم هذه التعليمة للبحث عن شيء محدد في الـ memory

Example :-

Search on 'W' letter if it found store 3 in CL reg ?

Using String Instruction

Arr DB 'Hello - World'
1 2 3 4 5 6 7 8 9 10 11

Sol :-

```
mov Cx, 11
mov Di, offset Arr
mov AL, 'W'
REPNE ScasB ;  → يعني شغال طول ZF=0
JNZ L1 ;  بعد ما ينتهي من اللف ولم يجد 'W' يكون ZF=0
mov CL, 3  وبالتالي ينتهي البرنامج
L1 : HLT
```


Scanned by CamScanner

Scanned by CamScanner

Example :-

Find the number of 'b' in array x store it in AH Reg?

x DB 'abc bba cd ba'

Using string instruction

Sol :-

```
mov CX, 10
mov Di, offset x
mov AL, 'b'
L2: REPNE ScasB
inc AH          ; لما يلاقي b
Cmp CX, 0       ; حتى يروح ويدور
JNE L2
```

حل آخر باستخدام الـ Loop.

```
org 100H
mov Di, offset x
mov AL, 'b'
mov Cx, 10
L1:
Scasb
LoopNE L1
Add AH, 1
Cmp Cx, 0
JNZ L1
HLT
```

انتبه
أن Scasb تتعامل مع AL و Di فقط

Scanned by CamScanner

Scanned by CamScanner

Example:- Search 'Wo' in 'Hello World'

Sol:-

```
mov Cx , 6        ← هي عدد الحروف نحط 5.5 لكن  → 5 < 5.5 < 6
mov Di , offset x ← دائماً نقرب للأكبر          (6 اكبر)
mov Ax , 'Wo'     ← لو كان يبحث عن 'Lo'
REPNE Scasw       في حالة الـ word
JNZ L1            ما راح يلاقيها لذلك
mov Cl , 3        نستخدم ScasB
L1 : HLT
```

← رح ناخذ عليها مثال.

Example :- Search 'Lo' in 'Hello World'

Sol:-

```
mov Di , offset x
mov AL , L
mov Cx , 11
L :
   Scasb
   Loopne L
cmp [Di] , 'o'
   JZ L2
Cmp Cx , 0
JNZ L
Jmp L3
L2 : Add Y , 1
L3 : HLT
```

Cmp Cx , 0
JNZ L
← حتى اتأكد انه ما فيه 'Lo'

← يبحث عن 'L' اذا وجدها يخرج من الـ loop

← يبحث عن 'o' اذا وجدها يعمل JZ

← تكملة الحل

Scanned by CamScanner

Scanned by CamScanner

CMPS Instruction.

يعمل مقارنة بين memory مع memory

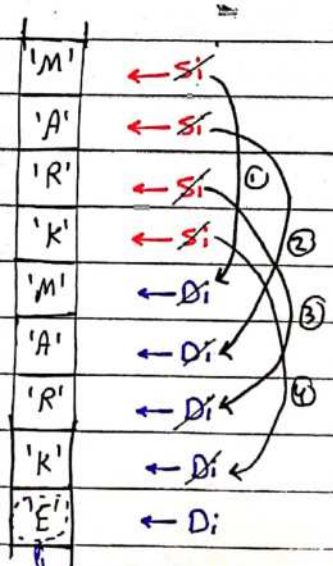
* CMPSB
Compare Byte
[Di] mem. ⟷ [Si] mem.

* CMPSW
Compare Word
[Di] mem. ⟷ [Si] mem.

DF		DF	
DF = 0	DF = 1	DF = 1	DF = 0
inc Si	Dec Si	Sub Si, 2	Add Si, 2
inc Di	Dec Di	Sub Di, 2	Add Di, 2

Ex :-

```
X DB 'MARK'
Y DB 'MARKE'
CLD
LEA Si, X
LEA Di, Y
mov Cx, 5
REPE CmpsB
JNZ END
mov AL, 3
END: HLT
```



اذا كان بدل E ← M ، راح يصير الشرط equal وبذلك يتخلص البرنامج

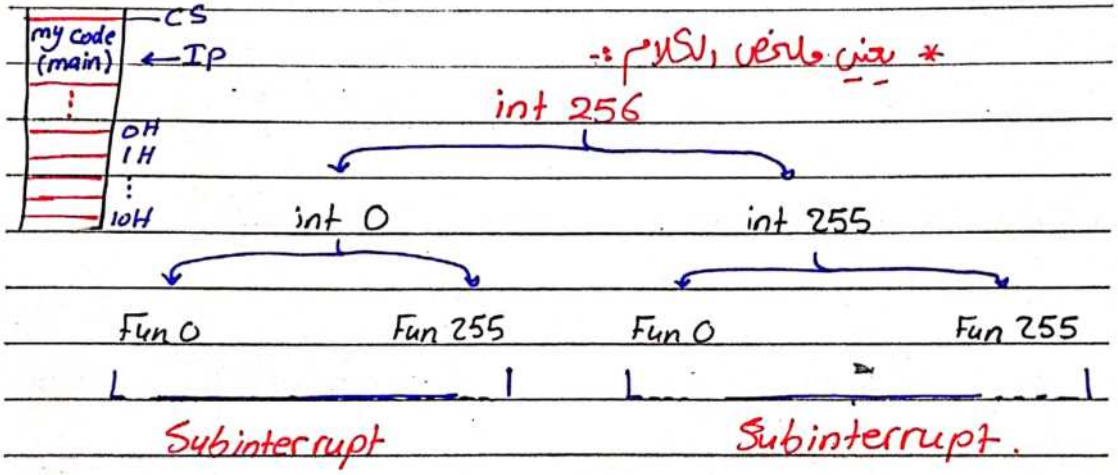
JCXZ
Jmp if Cx=0

Scanned by CamScanner

Scanned by CamScanner

* Interrupts :-

في هذا الدرس راح نتعرف على مجموعة تعليمات (interrupts) ، كل واحدة من هذه التعليمات تقوم بوظائف معينة.
ويوجد لدينا interrupt 256 (int) وتنقسم كل واحدة من ال int إلى Function 256 (Fun) ، كل (Fun... + int...) تقوم بعملية محددة.

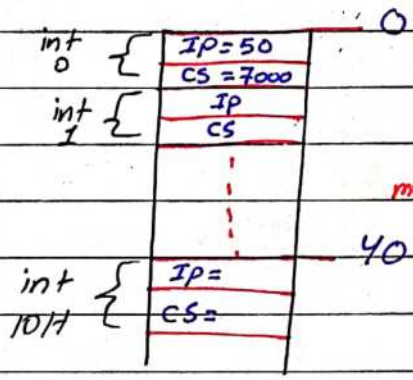


* يعني ماكو داعي للكلام :-

* int Vector

* لتحديد موقع ال int

int. Value * 4 = int. Address



* هيسا كل int تحتوي على CS و IP

IP مؤشر في الـ mem
CS عنوان في الـ mem

* عندما نصل في الكود عند تعليمات int بعمل إلنا مع Push



Scanned by CamScanner

احنا حكينا انه عندي int 256 ، رح نوخد منهم 2 فقط.

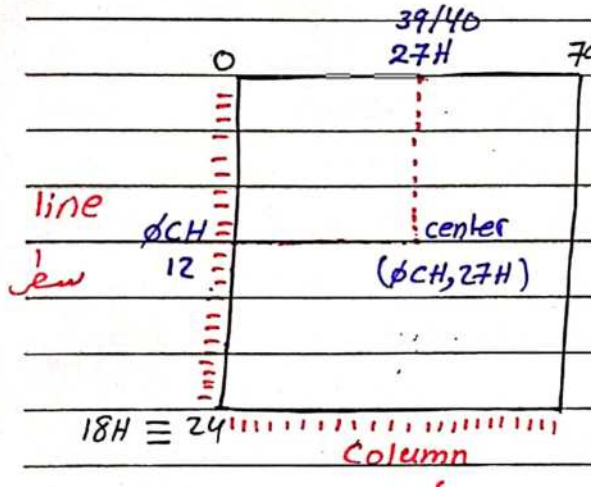
وحكينا انه ال int بواحد بينقسم إلى Fun 256 ال نوخد منهم فقط

INT 10H
- Fun AH = 02H
- Fun AH = 06H
- Fun AH = 07H
- Fun AH = 09H

INT 21H
- Fun AH = 01H
- Fun AH = 02H
- Fun AH = 09H
- Fun AH = 0AH

* نحن نقوم بتخزين ال Fun في AH دائماً

* تتعامل تعليمات ال interrupt مع ال Screen بشكل أساسي
وال Screen عبارة عن :-



* تتكون كل Screen من 8 صفحات
يكون ترقيمهم من 7 0
وتتكون كل صفحة ايضاً من 24 سطر
و 79 عمود.

ملاحظة مهمة جداً :-
دائماً نكتب ال Parameters قبل ال int. Value ونفهم الترتيب في المكتبة
ال Parameters ، اكيد ان نكتب قبل ال int. Value

Scanned by CamScanner

Scanned by CamScanner

* INT 10H / Function 02H
  ↳ Set cursor ↳ تخزن قيمة الـ Fun في AH دائماً

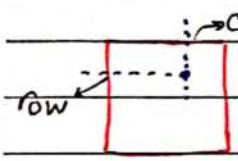
- تقوم هذه التعليمة بتحريك مؤشر الكتابة عن طريق تحديد السطر والعمود.

- Parameters :-

1. BH → Number of page
   * نحدد رقم الصفحة التي سوف نعمل عليها، مثال:- Mov BH,2 ⇐ ح يشتغل على الصفحة الثانية.
   * اذا لم نحدد، يكون ذلك لكن ح يشتغل على الصفحة (Zero)

2. DH → Number of row
   * نحدد رقم السطر.
   مثال:- Mov DH,3 ⇐ ح يشتغل على السطر الثالث.

3. DL → Number of Column
   * نحدد رقم العمود.
   مثال:- Mov DL,5 ⇐ ح يشتغل على العمود الخامس



Example :- Set Cursor at center of Screen ? السؤال يريد أن نحرك المؤشر لمنتصف الشاشة

Sol:-
```
mov AH, 02H ; Fun رقم الـ حددت
mov DL, 00H ; رقم الصفحة
mov DH, 0CH ; رقم السطر
mov DL, 27H ; رقم العمود
int 10H
```

لازم نحط أول شيء الـ Parameters وبعدين الـ int
بصير اختصر
mov Dx, 0C27
(DH = رقم السطر ، DL = رقم العمود)

Scanned by CamScanner

Scanned by CamScanner

* INT 21 H / Function 01 H → AH=01H تخزن في
↳ Input one byte from keyboard

* هذه التعليمة تقوم بادخال عنصر واحد فقط عن طريق الـ Keyboard
* القيمة التي ادخلها من الـ Keyboard تخزن دائماً في AL

- parameters:- parameters تطلب

← يعني

Example:-
```
mov AH,01H
int 21H
```
] ح تطلع شاشة تستنى مني أدخل حرف واحد فقط من الـ Keyboard

A↵

مثلاً ادخلت حرف A وبعدها كبست Enter & يظهرلي الشاشة حرف A وتخزن في AL=A.

* INT 21 H / Function 02H → AH=02H تخزن في
↳ Display one byte on screen

* هذه التعليمة تقوم بادخال عنصر واحد فقط عن طريق الكود نفسه

- Parameters:-

* DL → Byte to print هون بتخزن العنصر اي بدي اطلعه كـ screen

Scanned by CamScanner

Scanned by CamScanner

Example :-

```
mov AH, 02H
mov DL, 'A' ; العنصر اي ع يطبع كال Screen
int 21H
```

A

* تستخدم هذه التعليمات ايضاً في int21H/AH=02

① mov DL, 09H → Tab (فراغ)
② mov DL, 0AH → Newline (سطر جديد)
③ mov DL, 0DH → CR (يرجع اي بداية السطر)

Example :- print hello in the center of Screen ?

```
Sol:- X DB 'hello'          بده مني اطبع كلمة hello في منتصف الشاشة
      org 100H              ⊛ يعني ع احتاج أول ما المؤشر يعدين يطبع حرف حرف
      mov Si, 0
      mov Cx, 5
      mov AH, 02H ; رقم ال Fun     اول شي عملت كال
      mov DH, 0CH ; رقم السطر      بتحرك المؤشر
      mov DL, 27H ; رقم العمود     اي منتصف الشاشة
      int 10H
; --- Set Cureser Function        * ما حددت رقم الصفحة ، ع يعتبرها page(0)
L1: mov DL, XESiJ ; الحرف اللي ع اطبعه
      mov AH, 02H ; رقم ال Fun     بعدين عملت لوب ع
      int 21H ; interrupt          يتحرك كال X حتى
      inc Si                       اطبع الحرف وطبعاً
      Loop L1                      ع تخزن الحرف في DL
; ---- Display one byte in Screen
      HLT
```


Scanned by CamScanner

Scanned by CamScanner

Example:- print

h
e
l
l
o

بشكل Using Procedure ؟ و
عمودي

Sol:-

```
mov Si,0        ; المؤشر إلي ع يتحرك و
mov Cx,5        ; عدد الحلقة و
L1: mov DL,X[Si] ; تخزنهم حتى اطبع عالشاشة و
Call print      ; بستدعي ال proc لطباعة الحرف و
mov DL,0AH      ; بعمل سطر جديد و
Call print      ; بستدعي ال proc لطباعة السطر الجديد و
inc Si
Loop L1
HLT
Print Proc
mov AH,02H
int 21H
RET
ENDP
X DB 'hello'
```

عملت لوب ع
يطبع حرف حرف
وحطيت بين
كل حرف وحرف
أمر New line
وطبعته كل مرة

ملاحظة:-
أمر سطر جديد ولازم و CR
هم ايضاً يحتاجوا $ بقيمة طباعة
وهي موجودة في مكتبة int 21H
AH = 02H

Scanned by CamScanner

Scanned by CamScanner

Example:- ادخل رقمين من الـ Keyboard وخزنهم برقم ثالث واطبع ناتج الجمع
$N_3 = N_1 + N_2$

Sol:-

```
mov AH, 01H      ]→ قراءة حرف من الـ Keyboard
int 21H          ]   ويخزن في 'AL'
And AL, 0FH      ; بدي احولها من '14' اي 4
mov N1, AL
int 21H          ; ادخل رقم ثاني
And AL, 0FH      ; بدي احول منه '12' اي '2'
mov N2, AL
Add AL, N1
or AL, 30H       ; بدي ارجع اي Char
mov N3, AL
mov DL, AL       ] بدي اطبع بعدين
mov AH, 02H      ]
int 21H          ] N3 = N1 + N2
                        14  12
```

لأن DL بطبع Char

Scanned by CamScanner

Scanned by CamScanner

INT 21 H / Function 09 H

↳ Display message → AH = 09 H

* هذه التعليمة تقوم بإدخال message كاملة ، يعني بدخل اكثر من حرف وبالآخر بنتهي المسج بـ $

- parameters :-

DX → offset of message

مثال :-

```
mov AH,09
mov Dx, offset X
int 21 H
HLT
X DB 'hello$'
```

Note :-

The message ended by $

يعني البرنامج عارف بنتهي عملية المسج لما يلاقي $ اذا ما لقاها بضل بعمل

'hello' هيك اذا بطبع كلمة

Example :-

X DB 'hello$' or → X DB 'hello','$' or X DB 'hello','24H'

mov AH, 09 H → Fun تحدد رقم

mov Dx, offset X → تعمل offset للمسج

int 21 H

hello → الشاشة

h e l l o ← الشاشة

Example :-

X DB 'h',0AH,'e',0AH,'l',0AH,'l',0AH,'o',0AH,'$'

mov AH, 09 H

mov Dx, offset X

int 21 H

* بما انها جملة طويلة وتخزن في Dx اللي تحتوي كل DL فاذا راح يعمل سطر جديد

Scanned by CamScanner

Scanned by CamScanner

INT 10H / Function 06H → Scroll up
Function 07H → Scroll down } AH.

* هذه التعليمة تقوم بتحريك السطر إلى أعلى أو أسفل وتقوم هذه التعليمة أيضاً بعملية حذف
Scroll down — Scroll up

وعلة Scroll up إذ يتحرك السطر الأول إلى أعلى وبالتالي تنحذف علة hello



-parameters:-

[1] AL → Number of line to Scroll (AL → 0 Full screen).

* يحدد عدد السطر إلي بدي اعزلها إلى أعلى او أسفل ، اذا حكيت mov AL,0 ← هذا يعني إذ تمسح كل إلي حدته.

[2] BH → Attribute (colors)

* هون انا بحدد لون الكتابة ولون السطر ، فلة BH=71H
Background color (لون خلفية الكتابة) ← Foreground color (لون الكتابة)

* الألوان درجة عندي من الصفر إلى F ، BH=0FH F→white ..... 0→Black

BH=0F0 ← خارج الحسبة

[3] CX → starting point

* بحدد من اي سطر يبلش وعمود مثال:- mov CX,234F
سطر ← عمود

[4] DX → ending point

* بحدد لوين بدي أوقف عملية التحريك عند اي سطر وعمود مثال:- mov DX,0523
سطر ← عمود

Scanned by CamScanner

Scanned by CamScanner

Example:- اكتب برنامج يقوم بـ Clear Screen

Sol:-

```
mov AH, 06H
mov AL, 00 H
mov BH, 0FH
mov Cx, 00H
mov Dx, 144FH
```

Example:-

```
str DB 'abcdefg$'
mov AH, 09 H
mov Dx, offset str
int 21H ——→ تقوم بطباعة
mov AL, 2
mov AH, 7
mov Cx, 0001 H
mov Dx, 0203H
int 10H ——→ Scroll down two lines from area
            between elements (0,1)
            & element (2,3)
```

str في الـ screen

a	b	c	d	e	f	g

Final Output →

a				e	f	g
	b	c	d			

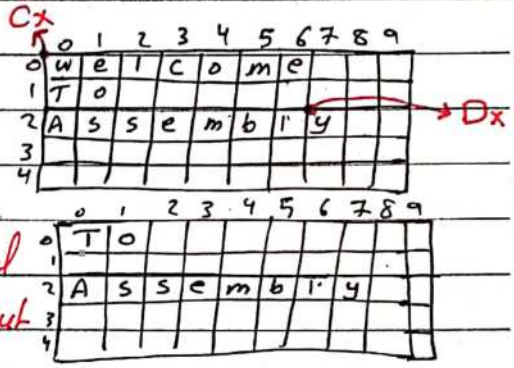
Scanned by CamScanner

Scanned by CamScanner

Example:- تتبع

```
msg DB 'welcome',0AH,0DH,'To',0AH,0DH,'Assembly','$'
mov AH,09H
mov DX,offset msg.
int 21H
mov AH,06H
mov AL,01H
mov BH,71H
mov CX,00H
mov DX,0106H
int 10
```

Sol:-



INT 21H / Function 0A.

↳ input from keyboard → يكتب أي شيء byte من

وهذا وهو احترازنا عن 01H

-parameters:-

① AH ⟶ Number function.

② Dx ⟶ offset of parameter list

ملاحظة:-

طريقة الادخال تختلف قليلاً، حيث يتم في البداية حجز مكان وتوضيح حجمه والحجم يكون اكتابة.

↳ لتوضيح ⇐

Scanned by CamScanner

Scanned by CamScanner

* توضيح الادخلة :-

ABC DB max ? max Dup (?)

- max الاول: عدد الـ byte اللي بدي احجزه
- ? الثاني: البرنامج لحاله بعد عدد الـ byte اللي دخلتهم
- max الثالث: هنا التعريف انه حجزت قيمة الـ max بالـ mem. يعني max byte بكبسة طبعاً بدون enter
- Dup (?): وبحط اركتم فالأقل أنا احط (?) وهنا كبسة الـ enter عشان يكبسها الـ emulation

ملاحظة: الـ max كان 40 يكون 39 للكتابة وآخر byte للـ enter.

max	← بخزن عدد الـ byte اللي حجزتهم
Actual	← بخزن الـ byte اللي دخلتهم
	ع بخزن الاشي
	اللي دخلته.

Example:-

```
mov AH, 0AH
mov Dx, offset ABC
int 21H
ABC DB 30, ?, 30 Dup(?)
```

sol:-

Assembly ↵

بما انه دخلت Assembly فالـ emu. رح يدخل بمل الـ ? ← 8 عدد بخزن باكيبورد

30	← max الـ عدد
8	← اطول الكلمة
A	
S	
S	
e	
m	
b	
l	
y	
:	
↵	

Scanned by CamScanner

Scanned by CamScanner

Sheet For Interrupt :-

[1] INT 10H :

① int 10H / Fun 02H

parameters :-

1- BH → Number of page
2- DH → Number of row
3- DL → Number of column.

② int 10H / Fun 06H → Scroll up
Fun 07H → Scroll down.

-parameters :-

1- AL → Number of line to scroll (AL=0 → Scroll Full screen)
2- BH → Attribute (color)

BH = number number
1 2
Back ground color ↵ ↳ Fore ground color

0 → Black
1 → White

3- Cx → Starting Point
4- Dx → Ending Point

Scanned by CamScanner

Scanned by CamScanner

[2] INT 21 H :

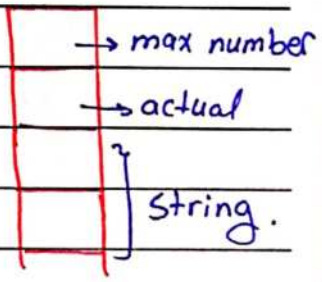
① int 21H / Fun 01H
↳ input one byte from keyboard
-parameters:-
null ⟶ لا يوجد
out : AL ⟶ your data (byte) ← يتخزن بها

② int 21H / Fun 02H
↳ Display one byte on Screen
-parameters :-
DL ⟶ Byte to print

oAH → New line
oDH → CR
ارجع كول إسطر

③ int 21 H / Fun 09H
↳ Display message
- parameters :-
Dx ⟶ offset of message
Note:- message ended by $

④ int 21 H / Fun 0A H
↳ Input stream
- parameters :-
Dx ⟶ offset of parameter list



Scanned by CamScanner

Scanned by CamScanner

Files

يوجد مجموعة من التعليمات للتعامل مع الـ Files ، فتح او اغلاقه او قراءة او كتابة
لكن قبل ان نتعرف على هذه التعليمات يجب ان نعلم امتداد الملف فما اذا يكون
Dos او txt او mp3

يعني ⇐ doc. اسم الملف
txt. اسم الملف
mp3. اسم الملف

* Int 21 / Fun 3CH : (Create File)

-parameters:-

[1] Cx : File attribute.
#0 : normal
#1 : read only
#2 : hidden

[2] Dx = offset of file name
if Cx=0 → every thing ok
if Cx=1 → there's error

[3] Ax : File handle

يتخزن الـ File handle في الـ Ax في حال CF = 0 ، يعني كل الامور تمام .
في حال كان في error يكون CF=1 وبالتالي يتخزن بـ Ax كود الـ error

* ملاحظة : اسم الملف لازم ينتهي بصفر مثال :- 'ABC.txt', 0

Scanned by CamScanner

Scanned by CamScanner

Ex:- h Dw ?

```
File name DB 'myfile.txt',0
mov AH, 3CH
mov Cx, 0
mov Dx, offset file name
int 21H
JC error
mov h, Ax
error: HLT
```

انشأ ملف

* int 21H / Fun 3DH (Open file)

① AL
- 0 : Read
- 1 : Write
- 2 : Read/Write ← غالبا نحط 2

if CF=0 ✓ , Ax = file handle
CF=1 ✗ , Ax = error

② Dx → offset of file name (ended by zero).

Scanned by CamScanner

Scanned by CamScanner

Example:- error افتح برنامج , اذا فتح طلع من البرنامج واذا ما فتح يطبع على شاشة كلمة

Sol:-

```
org 100H
mov AH, 3DH
mov AL, 2
mov Dx, offset Fname
int 21H
JC error
mov h, Ax
Jmp L2
error : Call print
L2: HLT
h DW ?
Fname DB 'File.txt', 0
msg DB 'error$'
print Proc
mov AH, 09
mov Dx, offset msg
int 21H
RET
print ENDP.
```


Scanned by CamScanner

Scanned by CamScanner

Int 21H / Fun 40H (Write Record)

1. Bx → File name
2. Cx → Number of bytes to write.
3. Dx → offset of data to write

if CF = 0 → ok
CF = 1 → error

Example:- اكتب برنامج بحيث انشئ ملف و اكتب عليه.

Sol:-

```
h DW ?
filename DB 'myFile.txt',0
Data DB 'hello'
mov AH,3CH
mov Cx,0
mov Dx, offset file name
Int 21H
JC error
mov h,Ax
mov AH,40H
mov Bx,h
mov Cx,5
mov Dx,offset data
Int 21H
```

Create File.

Write.

Scanned by CamScanner

Scanned by CamScanner

INT 21H / Fun 3FH (Read from file)

1. Bx → File handle
2. Cx → Number of byte to read.
3. Dx → offset of buffer

buffer :- مكان في الـ memory بخزن فيه عشان Data تتخزن فيها

Example:-

```
mov AH, 3FH      ← قبلها بقول open file.
mov Bx, h
mov Cx, 5
mov Dx, offset buffer
int 21H
JC error
   :
buffer DB 10 Dup(?)
```

welcome to assembly

Buffer
w
e
l
c
o
:

بخزن بايت بايت في Cx

INT 21H / Fun 3EH (closed file)

Bx → File handle

if

CF = 0 → ok → Ax = disptroyed

CF = 1 → error → Ax = error

Scanned by CamScanner

Scanned by CamScanner

Example:- assembly Language اكتب برنامج يحسب عدد 'a' في البرنامج التالي:-

```
org 100H
mov Si, 17
mov AH, 3DH
mov AL, 2
mov Dx, offset Fname
int 21H
JC error
mov Handle, Ax
L2: mov AH, 3FH
mov Bx, Handle
mov Cx, 1
mov Dx, offset Data
int 21H
JC error
cmp Data, 'a'
JNZ L1
Inc Connt
L1: Dec Si
JNZ L2
mov AH, 3EH
int 21H
error : HLT
```

open file

write

closed file.

```
Fname DB 'File.txt',0
Handle DW ?
Connt DB 0
Data DB ?
```


Scanned by CamScanner

Scanned by CamScanner

Example:-

```
Org 100H                                   ; welcome to
mov AH, 3D                                 ; open file
mov AL, 2
mov Dx, offset file name
int 21H
JC error
mov h, Ax
mov Bx, h
mov Cx, 5                                  ; لاحظ بكود لقراءة خمسة Cx=5
mov Dx, offset buffer                      ; Read  يعني رح يقرأ خمسة أحرف بس
mov AH, 3FH                                ; ويخزنهم بال mem بمتغير اسمه buffer
int 21H
JC error
mov buffer[5], '$'                         ; بعد ما قرأنا بدي اطبعه
mov AH, 09H                                ; باستخدام int وبدي
mov Dx, offset buffer                      ; write  اتذكر انه لازم
int 21H                                    ; ينتهي بـ $
mov AH, 3EH                                ; closed file
mov Bx, h
int 21H
error : HLT
file name DB 'ABC.txt', 0
h DW ?
buffer DB 5 Dup(?)
```

w	Buffer
e	Buffer[1]
l	Buffer[2]
c	Buffer[3]
o	Buffer[4]

Scanned by CamScanner

Scanned by CamScanner